

Secure electronic voting system based on blockchain with dilithium authentication and zero-knowledge proof verification

Received: 27 February 2026

Accepted: 15 July 2026

Published online: 26 July 2026

Cite this article as: Alsuwat E. Secure electronic voting system based on blockchain with dilithium authentication and zero-knowledge proof verification. *Sci Rep* (2026). <https://doi.org/10.1038/s41598-026-62925-1>

Emad Alsuwat

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

Secure Electronic Voting System Based on Blockchain with Dilithium Authentication and Zero-Knowledge Proof Verification

Emad Alsuwat ¹

¹ Department of Computer Science, College of Computers and Information Technology, Taif University, Taif 26571, Saudi Arabia, Alsuwat@tu.edu.sa ;

***Corresponding Author:** Emad Alsuwat. Email: Alsuwat@tu.edu.sa ;

Abstract: Electronic voting has become an important digital governance mechanism for remote elections, institutional decision-making, shareholder voting, public consultations, and large-scale Internet-based democratic participation. Despite its growing relevance, secure electronic voting remains difficult to implement because a practical system must simultaneously preserve voter anonymity, verify voter eligibility, prevent double voting, ensure ballot integrity, support public auditability, and maintain acceptable transaction throughput. To address these challenges, this study proposes a post-quantum secure and privacy-preserving blockchain-based electronic voting framework that integrates Dilithium digital signatures, zero-knowledge proofs, nullifier-based double voting prevention, encrypted ballot submission, smart contract-based election rule enforcement, and a Byzantine fault-tolerant consensus mechanism. In the proposed architecture, Dilithium signatures are used for post-quantum authentication of voter transactions and validator messages, whereas the zero-knowledge proof layer is used separately to verify voter eligibility, candidate validity, credential ownership, and correct nullifier generation without revealing the voter identity or ballot choice. Dilithium verification is performed externally at the transaction authentication layer, while the zero-knowledge circuit handles privacy-preserving voting logic. Each voter locally generates a private credential and submits only a public commitment during registration, thereby reducing the risk of authority-based impersonation or identity-to-vote linkage. Smart contracts verify the proof, reject reused nullifiers, enforce voting rules, and record auditable election events on the permissioned blockchain ledger. Experimental evaluation demonstrates that the proposed prototype achieves an average throughput of 408 transactions per second and an average block finalization time of 2.18 seconds under stress testing. The results indicate that the framework can provide a practical balance between post-quantum security, privacy preservation, verifiability, and transaction efficiency in permissioned electronic voting environments.

Keywords: Secure electronic voting, blockchain voting system, Dilithium digital signature, zero-knowledge proof, post-quantum authentication, nullifier-based double voting prevention.

1. Introduction

Electronic voting has become an important digital governance tool in today's democratic and institutional decision-making processes. Electronic voting systems are being discussed for voting at a distance, for voting in shareholder elections, for voting in university elections, for voting in organizational elections, for voting at public consultations, and for voting in large-scale Internet-based voting [1]. There are several benefits to electronic voting over traditional paper voting, such as faster collection of votes, lower administrative expenses, increased geographical access for voters, and the ability to audit the vote in real time. But the shift from

paper voting to Electronic Voting presents many security, privacy, and trust issues [2]. The electronic voting system should not only be accurate in counting the votes, but also satisfy the following requirements: only eligible voters should be allowed to vote; each voter should vote only once; the ballot should be confidential; the final result should be independently verifiable without violating the anonymity of the voters [3].

Electronic voting is not limited to national or local governmental elections. It is increasingly used in a wide range of digital decision-making scenarios, including remote political elections, institutional and university elections, shareholder voting, board and committee decisions, professional-association elections, trade and cooperative voting, public consultations, participatory budgeting, online referenda, community governance, and decentralized organizational decision-making. Internet-based voting platforms can improve accessibility, geographical participation, counting speed, administrative efficiency, and auditability. Such systems can be particularly useful for geographically distributed voters, overseas participants, individuals with mobility limitations, and organizations that require frequent or large-scale decision-making. However, the expansion of voting across Internet environments also introduces substantial security and privacy challenges. These challenges include remote voter authentication, identity exposure, ballot manipulation, double voting, coercion, compromised client devices, denial-of-service attacks, unauthorized database modification, insider threats, and the difficulty of achieving both ballot secrecy and public verifiability.

The main needs of an electronic voting system are security and privacy [4] [5]. Traditional digital voting systems typically use centralized authentication or traditional public-key cryptographic systems for voter authentication. While these methods can offer a simple way of verifying identity, they also have numerous shortcomings, such as use of authority [6], linkage to the voter [7], insider attack [8], single point of failure [9], and lack of transparency [10]. Furthermore, most of the existing systems use classical digital signature algorithms, such as RSA [11] or elliptic curve cryptography [12], that can be insecure in the presence of quantum computers on a large scale. The security of election infrastructures is of paramount importance, and with the continued development of quantum computing [13] [14], it is important to have cryptographic solutions that are quantum-resistant. This is especially critical for electronic voting since the records of the elections, voters' credentials, and audit trails might be required to be kept secure and verifiable for a number of years.

The decentralization, immutability, transparency, and auditability of blockchain technology [15] [16] have made it a promising option for establishing a secure electronic voting system. The events of an election, voting transactions, and the tallying of information can be recorded in a tamper-resistant way on a blockchain ledger. Smart contracts can also help automate the election rules, such as voter registration rules, ballot submission rules, rules for the voting period, nullifier checking, rules for publishing results, etc. Byzantine fault-tolerant consensus can offer quick finality and robustness in permissioned blockchain architectures against malicious or faulty validator nodes. But blockchain is not the end of the problems of electronic voting with regard to privacy and authentication. Blockchain transactions are typically transparent to the network users, making it easy to link the identity of the voter or their vote with the transaction. Voters' identity or their vote choice can be linked to the transaction if it is directly recorded on the blockchain, which can compromise the anonymity of the vote.

Zero-knowledge proofs are a useful tool for verifying the authenticity of an electronic voting system without revealing any information about the vote [17]. Using the zero-knowledge proof techniques, a voter can prove their eligibility, the validity of their credential, their selection of a valid candidate, and their ability to generate a voting nullifier without revealing their identity or their choice of candidate. Zero-knowledge proofs are especially appropriate for electronic voting systems, where it is important to be both publicly verifiable and private. A privacy-preserving identifier (called a nullifier) can be employed to guard against double voting, without revealing the voter's true identity. But the combination of zero-knowledge proofs with blockchain voting should be done wisely. Mixing authentication, eligibility checks, and ballot validation into a single cryptographic layer without any separation can make the system inefficient, hard to audit, and prone to privacy leakage in the implementation itself.

The next big challenge for the future of digital infrastructures is post-quantum cryptography [18]. Dilithium is a highly discussed lattice-based digital signature scheme, as it is secure, has efficient verification time, and is appropriate for digital authentication in quantum-resistant applications. Dilithium signatures can be used to verify voter transactions and validator communications without classical cryptographic assumptions in the context of electronic voting. But Dilithium signatures are not meant to be used to offer ballot privacy in isolation. As such, they need to be coupled with privacy-preserving techniques like zero-knowledge proofs.

Although eVoting has made significant strides in the fields of blockchain voting, post-quantum cryptography, and zero-knowledge verification, there are still several drawbacks in current electronic voting systems. Most of the voting systems based on blockchain are based on classical cryptographic authentication, which may not be quantum-resistant in the long-term. Some privacy-preserving voting schemes are not able to separate the authentication of the transactions from the zero-knowledge verification of eligibility, resulting in architectural confusion and less clarity of implementation. Issuing credentials centrally could present issues of linkage and impersonation. Fourth, the proposed voting systems are mostly concerned with theoretical security properties, while offering a limited performance analysis under realistic transaction stress. The security, anonymity, verifiability, double voting prevention, and throughput are the key requirements for a practical electronic voting system, and these are yet to be solved.

To overcome these problems, this paper presents a secure electronic voting system that uses blockchain technology, Dilithium authentication, and zero-knowledge proof verification. The main contributions of this study are fourfold:

- (i). This paper presents an electronic voting system based on blockchain technology that incorporates the Dilithium digital signature for post-quantum secure authentication of voters' transactions and validators' communication. The proposed design is also more resilient to quantum attacks compared with the widely used conventional blockchain voting systems, which are mainly based on signatures using RSA or elliptic curves.
- (ii). The proposed framework is an architectural separation between the transaction authentication based on Dilithium and the verification of the vote based on zero-knowledge proof. Here, Dilithium is used to prove that transactions have been submitted and are valid, while the zero-knowledge circuit is used to prove that the voter is eligible, that the voter owns the credential, that the candidate is valid, and that

the voter generated the nullifier correctly without revealing the voter's identity or their ballot selection.

- (iii). The proposed mechanism is based on nullifiers to avoid duplicate voting while maintaining the anonymity of the voters. Every voter creates their own credential and only provides a public commitment when registering to vote, which decreases the possibility of private credential misuse by the authority, decreases identity-to-vote linkage, and prevents double voting without revealing the identity of the voter.
- (iv). The proposed system will implement a set of smart contracts to ensure election rules are adhered to, verify zero-knowledge proofs, prevent re-used nullifiers, document verifiable election events, and enable transparent ballot processing on a permissioned blockchain ledger. Experimental stress testing shows that this is both practically feasible and achieves an average transaction throughput of 408 transactions per second, with an average block finalisation time of 2.18 seconds, providing a good balance of security, privacy, verifiability, and transaction efficiency.

The rest of this paper is organized as follows. The related work on electronic voting, blockchain voting, post-quantum authentication, and privacy mechanisms based on zero-knowledge proofs is presented in Section 2. In Section 3, the proposed system architecture is described, consisting of voter registration, generation of credentials, authentication using Dilithium, verification of zero-knowledge proof, generation of nullifiers, ballot encryption, execution of smart contracts, and consensus operation. The experimental setup, implementation details, performance metrics, and evaluation procedure are also included in Section 3. The experimental results, security analysis, and comparative interpretation are discussed in Section 4. Finally, Section 5 wraps up the study.

2. Literature Review

The literature reviewed shows a clear evolution of traditional architectures for blockchain voting systems towards privacy-preserving, verifiable, and post-quantum secure electronic voting systems. Previous research was mostly focused on decentralization, vote immutability, transparency, and fraudulent voting prevention. For instance, Ahn and Lee (2022) [43] designed a voting system on the Ethereum blockchain with smart contracts, while Priscilla et al. (2022) [47] suggested a voting workflow based on blockchain execution. Likewise, González et al. (2022) [42] explored the use of blockchain in enterprise-level secret ballots, and Faruk et al. (2022) [41] outlined the key security requirements and threats of blockchain voting. While these studies demonstrated the feasibility of blockchain-based election management, they offered only limited protection against attack by transaction-linkability, coercion, quantum computers, and powerful computers. The cost of public blockchain, exposure of voter identity, lack of scalability, and reliance on classical authentication methods were thus unsolved.

Later, the use of privacy-enhancing cryptographic methods gained some prominence. To achieve privacy while processing votes, Naidu et al. (2022) [46] used blockchain along with homomorphic encryption, whereas Wu and Kasahara (2023) [40] applied homomorphic encryption, zero-knowledge proofs, and smart contracts to verify votes in an encrypted manner. These studies marked a significant step toward cryptographically verifiable voting, a development that would mark a major paradigm shift from the traditional immutable vote storage to cryptographically verifiable voting. However, generating a homomorphic

encryption and a zero-knowledge proof can be very costly in terms of computation, storage, and communication, especially in large-scale public elections. The effectiveness of their work should be evaluated not only based on the theoretical security properties, but also based on the proof generation time, the time of verification, the consumption of gas in the blockchain, the number of transactions per second, and the compatibility of the device with the voter.

In 2023, research was also conducted on alternative blockchain architectures and permissioned blockchain architectures. Jain et al. (2023) [37] used Hyperledger Fabric to enable controlled participation and efficient execution of smart contracts, while Esposito and Choi (2023) [45] showed an implementation based on Algorand. These platforms can help minimize the transaction costs and throughput constraints that are typical issues with Ethereum. But Jain et al. [37] did not include ZKP or post-quantum signatures, and Esposito and Choi [45] did not provide detailed explanations of the advanced voter-anonymity and unlinkability mechanisms. So, the change of the underlying blockchain platform enhances the efficiency of operation, but does not directly address the cryptographic flaws of voter authentication and ballot privacy.

The review-based studies offer more general evidence of the continuing shortcomings of blockchain voting. Vladucu et al. (2023) [38] discussed decentralization, privacy, security, and trust issues, and Almeida et al. (2023) [39] systematically analysed the implications of decentralization in electronic voting. The scalability requirements and open research issues were discussed by Jafar et al. (2022) [44], and the blockchain voting mechanisms and future design directions were discussed by Sharp et al. (2024) [28]. Similarly, Hajian Berenjestanaki et al. (2024) [29] emphasized the benefits of blockchain in terms of transparency, integrity, and privacy. These reviews always indicate that blockchain can help establish a tamper-evident election record, but blockchain transparency does not imply that ballots are secret, that they are not subject to coercion, that voters are anonymous, or that voters' identities are verified securely. In addition, most of the systems reviewed are still in a conceptual stage or are only laboratory prototypes and do not represent independent, audited electoral platforms.

There have been several studies in 2024 that were trying to enhance the technical architecture of the blockchain voting system. Wang et al. (2024) [32] designed a flexible blockchain voting system that has enhanced security properties, while Kafhali and Sudhakar (2024) [33] outlined the key features of secure, private, and auditable blockchain voting. Matzliach et al. (2024) [36] achieved voting by smart contracts on EOS, highlighting the possibility of voting on a specific platform. But requirement-oriented and platform-oriented contributions are not enough to solve the long-term cryptographic security issue. Unless they are redesigned to use classical public-key mechanisms, the majority of infrastructures, such as EOS, Ethereum, Hyperledger Fabric, etc., rely on classical public-key mechanisms. Therefore, the efficiency of the platform should not be taken as an indication of post-quantum security.

The concept of identity privacy was also explored in greater detail in the study of Marcellino et al. (2024) [34], where voters were authenticated using zero-knowledge identity proofs without their identities being disclosed directly. This is crucial because traditional election addresses might enable the tracking or correlation of election transactions. But if identity privacy is not tied to eligibility credentials, election-specific nullifiers, and secure spent-vote detection, then it is possible to vote repeatedly. Furthermore, the authentication transaction is still susceptible to quantum attacks when classical signatures are used for authorizing the

submission of proof. Majumder et al. (2024) [35] integrated ECC, Exonum blockchain, signatures, and zero-knowledge properties, but the requirement of ECC makes it less suitable in the long run, as quantum computers are becoming more powerful and could break ECC-based signatures. So, the study enhances the security of today's world, but it does not offer cryptographic longevity.

There are some interesting signs of progress in electronic voting moving to the post-quantum era in Joni et al. (2024) [31] and Hough et al. (2023) [48]. In order to achieve the above goals, Joni et al. [31] proposed the PQMPCHAC-Bchain architecture with the combination of post-quantum-secured multiparty consensus and token verification. This research is among the relatively small number of studies that explicitly acknowledge threats in the quantum era in the context of blockchain voting. It is difficult to confirm the claimed scalability and security because of its architectural complexity and lack of independent verification; however, under realistic election workloads. Hough et al. [48] introduced an efficient voting construction based on the lattice that is based on NTRU, which offers a more solid theoretical basis for quantum-resistant voting. However, the proposal was largely cryptographic and did not include any linkage to smart contracts on the blockchain, decentralized auditability, zero-knowledge voter verification, or practical election-management procedures. These restrictions suggest that post-quantum cryptography and blockchain voting have been mostly pursued as distinct research streams.

The 2025 studies show greater focus on resisting coercion, self-tallying, formal security, and a higher level of encrypted computation. Kho et al. (2025) [25] proposed a provably secure coercion-resistant voting system and formally evaluated the confidentiality, anonymity, unforgeability, and verifiability. The formal treatment builds confidence in the protocol-level claims, but the study was not focused on blockchain throughput, smart-contract costs, or integration of post-quantum signatures. ElSheikh et al. (2025) [26] proposed a system that uses homomorphic time-lock puzzles and ZK-SNARKs for self-tallying in a privacy-preserving manner. This removes or reduces the need for a central authority to count votes, but it can be a complex setup process, generate proof, and impose an overhead on the actual implementation on low-resource voter devices. Shi et al. (2025) [27] enhanced fairness, anonymity, and protocol flexibility, but the execution of the protocol was not a key feature of their design using blockchain. As such, these works have good cryptographic properties but cannot provide a solution for decentralized execution, post-quantum authentication, efficient zero-knowledge verification, and scalable election management. Blockchain voting has been proven to be a successful solution in controlled settings in application-oriented studies.

Marouan et al. (2025) [23] showed that an institutional election, in the form of a university election, is a suitable testbed for a decentralized voting system. In general, however, university elections have fewer voters, less political opposition, and less opposition pressure than national elections. The conclusions are thus not directly applicable to governmental elections. Similarly, Rahul (2025) [22] talked about the transparency and trust of blockchain-based voting, but the contribution lacked in-depth cryptographic and implementation details. These studies are useful in theory, but lack in terms of coercion resistance, leakage of privacy, quantum-safe authentication, and large-scale resilience. Elhoseny et al. (2025) [24] furthered the concept of vote privacy by using Paillier homomorphic encryption, hybrid additive homomorphic encryption, and smart contracts. The method ensures that vote storage and tallying are encrypted and enhances transparency and immutability. But without post-

quantum signature authentication, there is a security imbalance, and ballots can still be encrypted, but the transactions or credentials that are used to cast ballots can be quantum-attackable. Likewise, Galal et al. (2026) [20] proposed a dual-encryption blockchain design using AES and RSA for enhancing the privacy and transparency of blockchain. While symmetric AES can be rather resistant if proper key sizes are used, RSA authentication and key exchange are susceptible to Shor's algorithm. The system is therefore only considered to be privacy-enhanced and not post-quantum secure.

The recent surveys of Ohize et al. (2025) [21] and Revelo Sánchez et al. (2026) [19] have already demonstrated that the research on blockchain e-voting has grown significantly and has now incorporated the use of smart contracts, homomorphic encryption, blind signatures, and zero-knowledge proofs. Both of the reviews, however, suggest the majority of solutions are conceptual, simulation-based, or pilot deployments. While there has been good growth in publications, Ohize et al. [21] pointed out that scalability, deployment, authentication, and architectural security issues continue to be problematic; Revelo Sánchez et al. [19] showed that despite this growth, the adoption of the technology in a real-world context is still limited. This distinction is important because there are not necessarily more proposed frameworks, the more mature the technology.

The number of independent audits, adversarial testing, usability studies, formal verification, regulatory compliance, and large-scale election trials is still relatively limited. An important socio-technical perspective was added by Mannonov and Myeong (2024) [30], who used the Technology Acceptance Model to look at citizen acceptance. Their results suggest that other factors, such as perceived usefulness, trust, transparency, and ease of use, also play a role in public acceptance of electoral adoption, in addition to technical security. However, perception studies are unable to confirm the cryptographic security of a system. A voting platform can be trusted by the public, but still has vulnerabilities; it can be technically secure, but not accepted due to a lack of understanding of how it operates. Thus, the research on blockchain e-voting needs to take into account various aspects of cryptographic assurance, usability of the system, institutional accountability, and public verifiability. A summary of the literature reviewed is presented in Table 1.

Table 1: The literature review of Secure Blockchain-Based Electronic Voting, Zero-Knowledge Verification, and Post-Quantum Authentication Studies from 2022 to 2026

Ref.	Author and Year	Main Method Used	Security / Privacy Mechanism	Reported Results / Key Findings	Main Limitation
[19]	Revelo Sánchez et al., 2026	Systematic review of blockchain-based e-voting systems	Blockchain, smart contracts, ZKP, blind signatures, homomorphic encryption	Reviewed 2022–2025 studies and identified strong growth in blockchain e-voting research	Mostly conceptual and pilot-level studies; limited real-world deployment
[20]	Galal et al., 2026	Blockchain-based privacy-enhanced e-voting	AES, RSA, blockchain transparency	Proposed a dual-encryption model to enhance vote privacy and transparency	Uses classical cryptography; not quantum-resistant
[21]	Ohize et al., 2025	Survey of blockchain-secured e-voting architectures	Encrypted voting, blockchain auditability, and authentication	Provided architectural taxonomy and identified security, scalability, and deployment challenges	Survey-oriented; does not implement a new PQ voting protocol
[22]	Rahul, 2025	Articulation of blockchain-enabled e-voting systems	Decentralized ledger, immutable storage, smart contracts	Discussed the transparency and trust benefits of blockchain voting	Practical implementation and cryptographic depth are limited
[23]	Marouan et al.,	Blockchain-based	Blockchain record	Demonstrated suitability	Limited to a university

	2025	university election system	keeping, authentication	for institutional voting contexts	setting; scalability and PQ resistance not addressed
[24]	Elhoseny et al., 2025	Efficient blockchain voting with hybrid encryption	Paillier homomorphic encryption, HAHE, smart contracts	Improved privacy, immutability, and transparency using encrypted vote storage	Does not include post-quantum signature authentication
[25]	Kho et al., 2025	Provably secure coercion-resistant e-voting	Confidentiality, anonymity, unforgeability, and CAI verifiability	Provided formal security analysis for coercion-resistant voting	Not focused on blockchain throughput or PQ signatures
[26]	ElSheikh et al., 2025	Self-tallying e-voting using time-lock puzzles and ZK-SNARKs	Homomorphic time-lock puzzles, ZK-SNARKs	Enabled self-tallying while preserving privacy and verifiability	Complex cryptographic setup; implementation overhead may be high
[27]	Shi et al., 2025	Efficient and flexible voting protocols	Fairness, anonymity, cryptographic voting protocol	Improved fairness, anonymity, and flexible protocol design	Not centered on blockchain smart-contract enforcement
[28]	Sharp et al., 2024	Survey and proposal of blockchain e-voting mechanisms	Blockchain, authentication, verification	Reviewed blockchain e-voting mechanisms and proposed design directions	Proposal-level analysis; limited stress testing
[29]	Hajian Berenjestanaki et al., 2024	Technology review of blockchain e-voting	Blockchain frameworks, consensus, privacy tools	Identified blockchain benefits in transparency, integrity, and privacy	Scalability and practical deployment remain under-addressed
[30]	Mannonov and Myeong, 2024	Citizen perception study using TAM	Blockchain trust, perceived usefulness, and transparency	Examined public acceptance of blockchain-based e-voting systems	Focused on perception rather than technical security
[31]	Joni et al., 2024	Hybrid blockchain-based EVM architecture	Post-quantum secured multiparty consensus, token verification	Proposed PQMPCHAC-Bchain for scalable and post-quantum-safe e-voting	Complex architecture requires broader independent validation
[32]	Wang et al., 2024	Efficient and versatile blockchain e-voting scheme	Blockchain, cryptographic verification	Designed a flexible blockchain voting scheme with improved security properties	PQ security and advanced ZKP integration are not central.
[33]	Kafhali and Sudhakar, 2024	Requirement analysis of blockchain e-voting	Security, privacy, auditability, blockchain	Defined the significance and requirements of blockchain-based e-voting	Requirement-focused; no full cryptographic prototype
[34]	Marcellino et al., 2024	ZKP-based identity authentication for e-voting	Zero-knowledge identity proof	Preserved voter identity privacy during authentication	Does not address post-quantum transaction authentication
[35]	Majumder et al., 2024	ECC-EXONUM-eVOTING scheme	Blockchain, zero-knowledge property, signature-based voting	Provided signature-based e-voting with blockchain and ZK property	ECC is not post-quantum secure
[36]	Matzliach et al., 2024	EOS blockchain-based e-voting	EOS blockchain, smart contracts	Demonstrated blockchain-based vote recording using the EOS platform	Platform-specific; limited PQ and ZKP support
[37]	Jain et al., 2023	Hyperledger Fabric-based e-voting	Permissioned blockchain, smart contracts	Proposed permissioned blockchain approach for secure e-voting	Does not integrate ZKP or post-quantum signatures
[38]	Vladucu et al., 2023	Survey of blockchain e-voting	Blockchain, decentralization, privacy, security	Reviewed blockchain e-voting systems and identified trust, privacy, and scalability issues	Review only; no new implementation.
[39]	Almeida et al., 2023	Systematic literature survey on decentralization in e-voting	Decentralized voting, blockchain, distributed trust	Analyzed the effects of electronic voting systems	Does not propose a new PQ/ZKP framework
[40]	Wu and Kasahara, 2023	Smart-contract e-voting with homomorphic encryption and ZKP	Homomorphic encryption, zero-knowledge proof, smart contract	Combined encrypted voting and ZKP for smart-contract-based verification	Computational cost and deployment scalability remain concerns
[41]	Faruk et al., 2022	Blockchain e-voting design and security requirements	Blockchain security design, threat analysis	Presented requirements, design, and security perspective	Does not use PQ signatures or nullifier-based ZKP logic
[42]	González et al., 2022	Enterprise blockchain e-voting system	Enterprise blockchain, secret ballot, reliability	Proposed reliable enterprise blockchain voting architecture	Limited privacy-preserving cryptographic depth
[43]	Ahn and Lee, 2022	Ethereum-based e-voting implementation	Ethereum, smart contracts, and fraudulent voting prevention	Implemented early Ethereum voting and evaluated adoption aspects	Public-chain cost and privacy limitations remain
[44]	Jafar et al., 2022	Systematic literature review and meta-analysis	Blockchain scalability, e-voting requirements	Synthesized scalable blockchain e-voting solutions and open issues	Literature-based; no new architecture
[45]	Esposito and	Algorand-based	Algorand blockchain,	Demonstrated design and	Limited focus on PQ

	Choi, 2023	blockchain e-voting	smart contracts	implementation on Algorand	cryptography and voter anonymity logic
[46]	Naidu et al., 2022	Blockchain with homomorphic encryption	Homomorphic encryption, blockchain	Addressed encrypted vote processing for privacy	Lacks post-quantum authentication and advanced ZKP
[47]	Priscilla et al., 2022	Decentralized e-voting using smart contracts	Blockchain, smart contracts	Proposed decentralized smart-contract voting workflow	Limited scalability/security depth
[48]	Hough et al., 2023	Lattice-based electronic voting from NTRU	Post-quantum lattice cryptography	Proposed a more efficient lattice-based voting for quantum-safe e-voting	Cryptographic proposal; not integrated with blockchain smart contracts

Research Gaps Identified

The literature shows that there are four main research streams: Privacy-preserving ballot processing with homomorphic encryption and zero-knowledge proofs [24], [26], [34], [35], [40], [46]; Post-quantum voting and consensus mechanisms [31], [48]; and Formal analysis of anonymity, fairness, coercion resistance and verifiability [25], [27]. But this is not the case with these streams, and they are not usually combined in one practical architecture. While blockchain-based voting systems are often vulnerable to quantum computers, existing blockchain-based systems are generally based on quantum security methods like ECC or RSA authentication, and blockchain-based voting systems are not always smart-contract-enabled, auditable, or scalable. Likewise, ZK authentication methods can obfuscate the identity of the voter, but not necessarily offer election-specific nullifiers, double-voting protection, on-chain verification of transactions, or efficient verification. Thus, there is a need for a fully integrated voting system capable of simultaneously offering post-quantum voter authentication, privacy-preserving voter eligibility verification, anonymous but unique ballot casting, double voting prevention by using a nullifier, encrypted or confidential tallying, and blockchain-based public audibility. An effective framework should also consider the cost of computation, the size of the proof, the size of the signature, the time required to verify the proof, the throughput, the storage requirement for the blockchain, the cost of smart-contract execution, the resistance to coercion and collusion, and the usability of the system in realistic election settings.

3. Materials and Methods

3.1 Research Design and System Objectives

In this study, we use a design and evaluation research approach to design a secure, privacy-preserving, auditable, and post-quantum-resistant electronic voting framework, as shown in Figure 1. The proposed system employs a permissioned blockchain, uses Dilithium digital signatures, verifies zero-knowledge proofs, submits ballots encrypted, implements double-voting prevention using nullifiers, adopts a smart-contract-based election management system, and adopts Byzantine fault-tolerant consensus. The main goal is to create a complete electronic voting system in which the eligibility of each voter can be verified without revealing who the voter is, the votes are kept confidential, no one can vote twice, and the votes are independently auditable by authorized stakeholders.

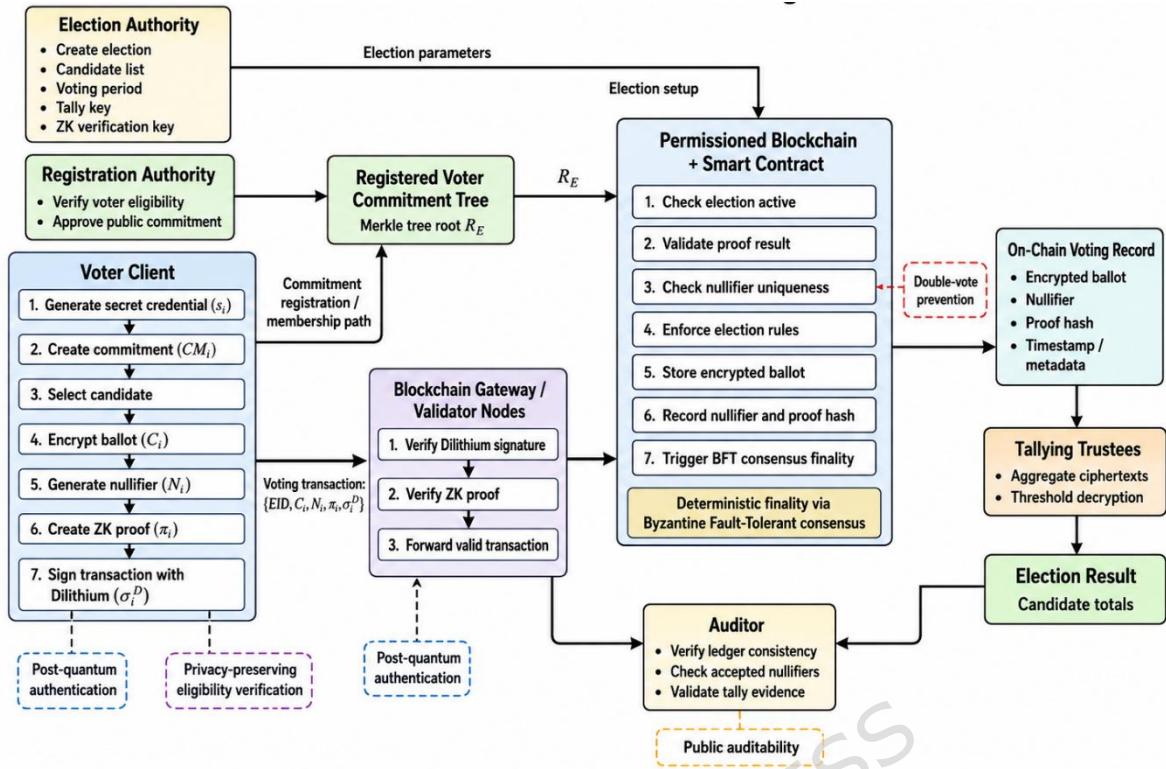


Figure 1: The overview of the proposed post-quantum secure and privacy-preserving blockchain-based electronic voting framework

One of the core design decisions is to separate eligibility verification (privacy) from transaction authentication (fundamental). Voters' transactions and validator communication are authenticated using Dilithium signatures at the blockchain transaction layer. Zero-knowledge proofs, on the other hand, can be used to check the conditions of the private vote, such as whether the voter has the credential, whether the voter is in the set of registered voters, whether the candidate is valid, whether the ballot is correct, and whether the nullifier is generated appropriately for the election. The separation allows avoiding unnecessary growth of the zero-knowledge circuit, while maintaining the authentication level at the communication and transaction level as being post-quantum resistant.

The proposed framework will meet the security and operational requirements as follows:

1. Only eligible and registered voters can cast valid ballots.
2. Each voter can submit no more than one accepted ballot in an election.
3. The identity of a voter cannot be linked to the selected candidate.
4. Ballots cannot be modified after transaction authentication and confirmation.
5. Election rules are enforced automatically through smart contracts.
6. Election events remain auditable without disclosing sensitive voting information.
7. Transaction and validator authentication remain resistant to quantum-enabled adversaries.
8. The system maintains acceptable throughput and block-finalization latency under increasing voting workloads.

3.2 System Entities

The proposed electronic voting environment consists of six main entities – the election authority, the registration authority, the voter, the blockchain validators, the election smart contract, and the auditor. These are detailed below in Table 2:

The election authority and the registration authority can be separated from an administrative point of view to decrease centralized trust. The registration authority checks voter eligibility, but only gets the public credential commitment from the voter. It has no way to get the secret of the private credential needed to build a valid voting witness. This means that the voter's private credential must be used to create a valid ballot for the voter by the registration authority; otherwise, the voter's private credential will be compromised.

Table 2. Principal entities and responsibilities in the proposed system

Entity	Primary responsibility	Accessible information	Restricted information
Election authority	Creates the election, candidate list, voting period, tallying policy, and election parameters	Election identifier, candidate identifiers, public cryptographic parameters, validator configuration	Voter credential secrets and plaintext ballots
Registration authority	Confirms real-world eligibility and approves public credential commitments	Voter identity during registration and the submitted public commitment	Private credentials, secret, and future ballot choice
Voter	Generates the credential, encrypted ballot, nullifier, proof, and authenticated transaction	Own credential, selected candidate, public election parameters, registration path	Credentials and ballot choices of other voters
Blockchain validators	Validate transactions, execute consensus, and finalize blocks	Signed transactions, zero-knowledge proofs, nullifiers, encrypted ballots, and public metadata	Voter identity and plaintext candidate selection
Smart contract	Enforces election rules and stores accepted voting events	Election status, candidate set, verification results, nullifier state, and encrypted ballot	Private witness values used in proof generation
Auditor	Verifies ledger consistency, participation records, and tally correctness	Public ledger, accepted nullifiers, encrypted ballots, proofs, and tally evidence	Identity-to-ballot relationship

3.3 Permissioned Blockchain Architecture

A permissioned blockchain is chosen because the electoral process demands that participation by validators be controlled, the finality of transactions be deterministic, the computational cost be predictable, and the institutions be accountable. The validators can be independent of each other, such as election administrators, judicial representatives, universities, accredited observers, civil society organizations, and authorized political representatives.

The blockchain only contains anonymous voting data. For every accepted voting transaction, there is an election identifier, an encrypted ballot, an election-specific nullifier, a zero-knowledge proof, a Dilithium signature, and non-sensitive transaction metadata. No personal voter identities are ever stored on-chain, nor are candidate selections stored in plaintext. A transaction that was issued by the voter i was a vote transaction.

$$T_i = (EID, C_i, N_i, \pi_i, PK_i^D, \sigma_i^D, t_i), \quad (1)$$

Where the EID denotes the unique election identifier; C_i denotes the encrypted ballot; N_i denotes the election-specific nullifier; π_i denotes the zero-knowledge proof; PK_i^D denotes the Dilithium public verification key or an authorized verification-key reference; σ_i^D denotes the Dilithium signature; and t_i denotes the transaction timestamp or nonce. The append-only blockchain ledger for election E is expressed as

$$\mathcal{L}_E = \{T_1, T_2, \dots, T_m\}, \quad (2)$$

In this, m is the number of accepted and completed ballot transactions. If a transaction is completed with a consensus agreement that is Byzantine fault-tolerant, then it would take two Byzantine failures to alter an existing transaction.

3.4 Cryptographic Preliminaries

The proposed framework employs four principal cryptographic components:

1. collision-resistant cryptographic hashing;
2. binding and hiding credential commitments;
3. post-quantum Dilithium digital signatures; and
4. Zero-knowledge proof generation and verification.

Let $H(\cdot)$ denote a collision-resistant hash function and $\text{Com}(\cdot)$ denote a binding and hiding commitment function. The Dilithium signature scheme is represented by the following algorithms:

$$(PK_i^D, SK_i^D) \leftarrow \text{Dilithium.KeyGen}(1^\lambda), \quad (3)$$

$$\sigma_i^D \leftarrow \text{Dilithium.Sign}(SK_i^D, M_i), \quad (4)$$

$$b_i^D \leftarrow \text{Dilithium.Verify}(PK_i^D, M_i, \sigma_i^D), \quad (5)$$

where λ denotes the security parameter, SK_i^D denotes the private signing key, PK_i^D denotes the public verification key, M_i denotes the signed message, and $b_i^D \in \{0,1\}$ denotes the verification result. The zero-knowledge proof system is represented as

$$(pp, vk) \leftarrow \text{ZK.Setup}(1^\lambda, \mathcal{C}), \quad (6)$$

$$\pi_i \leftarrow \text{ZK.Prove}(pp, x_i, w_i), \quad (7)$$

$$b_i^{\text{ZK}} \leftarrow \text{ZK.Verify}(vk, x_i, \pi_i), \quad (8)$$

where pp denotes the proving parameters, vk denotes the verification key, $\mathcal{C}$ denotes the voting relation or arithmetic circuit, x_i denotes the public input, w_i denotes the private witness, and π_i denotes the generated proof. The cryptographic notation used in this study is listed in Table 3.

Table 3. Cryptographic notation used in the proposed framework

Symbol	Description
λ	Cryptographic security parameter
EID	Unique election identifier
VID_i	Administrative identity of voter i , used only during registration.
s_i	Voter-generated private credential secret
r_i	Randomness is used in credential commitment
CM_i	Public credential commitment associated with voter i
R_E	Root of the registered-voter commitment tree
p_i	Merkle membership authentication path for voter i
v_i	Candidate selected by voter i
C_i	Encrypted ballot
N_i	Election-specific nullifier
π_i	Zero-knowledge proof
(PK_i^D, SK_i^D)	Dilithium public and private keys
σ_i^D	Dilithium digital signature
(PK_T, SK_T)	Election tallying public and secret keys.
$\mathcal{N}_E$	Set of previously accepted nullifiers for election E
f	Maximum number of tolerated Byzantine validators

n	Total number of validator nodes
-----	---------------------------------

3.5 Election Initialization

In the election initialization, the election authority sets the election identifier, candidate set, registration deadline, voting interval, tallying policy, validator list, zero-knowledge verification key, and election encryption key. The election is represented as:

$$\Theta_E = (EID, \mathcal{V}, t_s, t_e, R_E, PK_T, vk, \mathcal{B}), \quad (9)$$

where $\mathcal{V} = \{1, 2, \dots, K\}$ denotes the valid candidate set, t_s and t_e denote the voting start and end times, R_E denotes the registered-voter Merkle root, PK_T denotes the public tallying key, vk denotes the zero-knowledge verification key, and $\mathcal{B}$ denotes the authorized validator set. The smart contract accepts voting transactions only while the election is active:

$$\text{Active}(E, t) = \begin{cases} 1, & t_s \leq t \leq t_e, \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

After election contract deployment, it is impossible to change any election parameters (candidates, election identifier, election registration root, voting period) without an explicit authorization process provided by the governance.

3.6 Privacy-Preserving Voter Registration

At the registration stage, administrative eligibility is checked, and a voter-controlled credential is created. Voters' eligibility to vote in the real world is checked by the registration authority, based on an official electoral register or an institutional identity system. But the cryptographic voting credential is created by the voter in their local area. Each voter i chooses a secret credential at random:

$$s_i \leftarrow \mathbb{S}\{0,1\}^\lambda, \quad (11)$$

Together with independent commitment randomness:

$$r_i \leftarrow \mathbb{S}\{0,1\}^\lambda. \quad (12)$$

Then the voter makes the public credential commitment:

$$CM_i = \text{Com}(s_i; r_i). \quad (13)$$

Only CM_i is submitted to the registration authority after successful eligibility verification. The values s_i and r_i remain exclusively under the voter's control. All approved commitments are inserted into a Merkle tree:

$$R_E = \text{MerkleRoot}(CM_1, CM_2, \dots, CM_q), \quad (14)$$

where q denotes the number of registered voters. Each voter obtains or reconstructs a membership path p_i , enabling the voter to prove that CM_i belongs to the registered set without disclosing the commitment's position in the tree.

3.7 Ballot Encoding and Encryption

Each candidate will have a different candidate identifier. The candidate v_i chosen must satisfy

$$v_i \in \mathcal{V}. \quad (15)$$

The ballot for K candidates can be represented as a one-hot vector:

$$\mathbf{b}_i = (b_{i,1}, b_{i,2}, \dots, b_{i,K}), \quad (16)$$

subject to:

$$b_{i,j} \in \{0,1\}, \sum_{j=1}^K b_{i,j} = 1. \quad (17)$$

The solution to equation (17) is a unique solution that is chosen.

The public tallying key is used to encrypt the ballot locally:

$$C_i = \text{Enc}(PK_T, \mathbf{b}_i; \rho_i), \quad (18)$$

where ρ_i denotes fresh encryption randomness. The ciphertext C_i is stored on the blockchain, whereas the plaintext ballot remains inaccessible to validators, registration authorities, and public auditors. When homomorphic tallying is supported, accepted encrypted ballots are aggregated as

$$C_{\text{agg}} = \bigotimes_{i=1}^m C_i, \quad (19)$$

where $\bigotimes$ denotes the ciphertext aggregation operation. The final tally is obtained through authorized decryption:

$$T = \text{Dec}(SK_T, C_{\text{agg}}). \quad (20)$$

SK_T can be split up into several tallying trustees using threshold secret sharing in order to create a stronger network of trust for distributing the keys, so that no one can decrypt the sum of the keys or any individual ballot.

3.8 Nullifier-Based Double-Voting Prevention

A nullifier is a value that is unique for each voter and election, based on the voter's private credential, and is deterministic. It allows the smart contract to identify multiple votes cast by the same voter without identifying the voter.

The nullifier generated by voter i is:

$$N_i = H(s_i \parallel EID \parallel d), \quad (21)$$

where d denotes a fixed domain-separation value associated with the nullifier function.

The election identifier makes sure that the same credential generates distinct nullifiers for different elections:

$$EID_a \neq EID_b \Rightarrow H(s_i \parallel EID_a \parallel d) \neq H(s_i \parallel EID_b \parallel d), \quad (22)$$

Except with negligible probability. The smart contract maintains the set:

$$\mathcal{N}_E = \{N_1, N_2, \dots, N_m\}, \quad (23)$$

Including all the previously accepted nullifiers. A ballot can only be accepted if

$$N_i \notin \mathcal{N}_E. \quad (24)$$

Once successful authentication and verification of the proof is completed, the smart contract modifies the nullifier set as:

$$\mathcal{N}_E \leftarrow \mathcal{N}_E \cup \{N_i\}. \quad (25)$$

Because N_i is verified within the zero-knowledge relation, a malicious voter cannot provide an arbitrary nullifier to evade duplicate-vote detection.

3.9 Zero-Knowledge Voting Relation

The zero-knowledge circuit ensures voters' eligibility, ownership of credentials, correctness of nullifiers, and validity of ballots, while not revealing the voter's credential, commitment position, membership path, or chosen candidate.

The public input is defined as:

$$x_i = (EID, R_E, N_i, C_i, PK_T, \mathcal{V}), \quad (26)$$

Whereas the private witness is defined as:

$$w_i = (s_i, r_i, CM_i, p_i, b_i, \rho_i). \quad (27)$$

The circuit verifies the following constraints:

$$CM_i = \text{Com}(s_i; r_i), \quad (28)$$

$$\text{MerkleVerify}(CM_i, p_i, R_E) = 1, \quad (29)$$

$$N_i = H(s_i \parallel EID \parallel d), \quad (30)$$

$$C_i = \text{Enc}(PK_T, b_i; \rho_i), \quad (31)$$

$$b_{i,j} \in \{0,1\}, \forall j \in \{1, \dots, K\}, \quad (32)$$

and

$$\sum_{j=1}^K b_{i,j} = 1. \quad (33)$$

The complete voting relation is expressed as:

$$\mathcal{R}_{\text{vote}}(x_i, w_i) = 1, \quad (34)$$

if and only if all constraints in Equations (28)– (33) are satisfied.

The voter generates the proof as:

$$\pi_i = \text{ZK.Prove}(pp, x_i, w_i). \quad (35)$$

The proof is verified using:

$$\text{ZK.Verify}(vk, x_i, \pi_i) = 1. \quad (36)$$

The zero-knowledge property guarantees that the verifier only learns that the voter has a valid registered credential, that he has created the correct election-specific nullifier, and that he has encrypted a valid candidate selection. The verifier does not know who the voter is, what the voter's credential secret is, which position the voter committed to, what path the voter followed to join the group, what the randomness is for the voter's credential, or what candidate the voter picked. The values of the public and private are provided in Table 4.

Table 4. Public and private values used in zero-knowledge verification

Category	Values	Visibility
Election parameters	$EID, \mathcal{V}, PK_T$	Public
Registration state	R_E	Public
Voting state	N_i, C_i	Public
Verification evidence	π_i	Public
Credential information	$s_i, r_i, \text{commitment opening}$	Private
Membership evidence	Merkle path p_i	Private
Ballot information	$b_i, \text{candidate choice } v_i$	Private
Encryption randomness	ρ_i	Private

3.10 Dilithium-Based Post-Quantum Authentication

Dilithium is not part of the zero-knowledge circuit, but rather is used to authenticate the external voting transaction. The advantage of this design is that it does not require a lot of circuit overhead to represent the Dilithium verification as arithmetic constraints. The authenticated transaction message is formed as follows:

$$M_i = H(EID \parallel C_i \parallel N_i \parallel H(\pi_i) \parallel t_i). \quad (37)$$

The voter signs the message using the Dilithium secret key:

$$\sigma_i^D = \text{Dilithium.Sign}(SK_i^D, M_i). \quad (38)$$

The blockchain gateway or validator verifies the signature using

$$\text{Dilithium.Verify}(PK_i^D, M_i, \sigma_i^D) = 1. \quad (39)$$

The signature is used to secure the encrypted ballot, nullifier, proof hash, election identifier, and transaction metadata. If the values are tampered with in any way, they will cause signature verification failure. If an adversary changes the ZKP without changing the signature, it will be detected by adding $H(\pi_i)$ to the signed message. Dilithium signatures are also used to authenticate validator-to-validator consensus messages. The consensus message m_j^{con} is signed by validator j as:

$$\sigma_j^{\text{con}} = \text{Dilithium.Sign}(SK_j^D, m_j^{\text{con}}). \quad (40)$$

This mechanism prevents unauthorized forgery of this information: block proposals, prepare messages, commit messages, validator votes, and checkpoint information. The local vote-generating function is given by

$$(C_i, N_i, \pi_i, \sigma_i^D) = \text{VoteGen}(\Theta_E, s_i, r_i, p_i, v_i, SK_i^D). \quad (41)$$

The voting transaction submission procedure algorithm is presented in Algorithm 1.

Algorithm 1: Voting Transaction Submission Procedure

1. The voter retrieves the current election parameters and the approved registration root.
2. The voter selects a candidate and constructs a valid one-hot ballot vector.
3. The ballot is encrypted using the public tallying key.
4. The election-specific nullifier is generated from the private credential.
5. A zero-knowledge proof is generated from the private witness and public election inputs.
6. The transaction message is constructed and signed using the Dilithium secret key.
7. The complete transaction is submitted to a blockchain gateway.
8. The gateway or validators authenticate and preliminarily validate the transaction.
9. The smart contract verifies the election status, zero-knowledge proof, and nullifier uniqueness.
10. The accepted transaction is ordered and finalized through Byzantine fault-tolerant consensus.

3.12 Smart-Contract Verification and Election-Rule Enforcement

The election smart contract is the part that provides deterministic policy enforcement. It checks the validity of the election period, transaction structure, candidate configuration, proof validity, transaction authentication, and nullifier freshness. The predicate of the ballot acceptance is:

$$\text{Accept}(T_i) = \begin{cases} 1, & \begin{aligned} &\text{Active}(E, t_i) = 1 \\ &\wedge b_i^D = 1 \\ &\wedge b_i^{\text{ZK}} = 1 \\ &\wedge N_i \notin \mathcal{N}_E \end{aligned} \\ 0, & \wedge \text{FormatValid}(T_i) = 1, \\ & \text{otherwise.} \end{cases} \quad (42)$$

On different blockchain platforms, Dilithium verification can be performed either before invoking the smart contract. The valid result is then passed to the environment where the contract is executed. The verification of zero-knowledge proof is done independently with the registered verification key. The smart contract validation rules are given in the table. 5.

Table 5. Smart-contract validation rules

Validation rule	Verification condition	Rejection reason
Election existence	EID corresponds to a deployed election	Unknown election
Voting period	$t_s \leq t_i \leq t_e$	The election is not active
Transaction authentication	Dilithium verification returns 1	Invalid transaction signature
Proof verification	Zero-knowledge verification returns 1	Invalid eligibility or ballot proof
Nullifier freshness	$N_i \notin \mathcal{N}_E$	Duplicate voting attempt
Ballot structure	Ciphertext and proof inputs are well-formed	Malformed ballot
Election binding	Proof and signature contain the same EID	Cross-election replay
State consistency	Registration root equals the approved election root	Invalid eligibility set

When a ballot is accepted, the smart contract records the event:

$$EV_i = (EID, N_i, C_i, H(\pi_i), h_{\text{block}}, t_i), \quad (43)$$

where h_{block} identifies the finalized block. The smart contract does not store the voter identity, private witness, credential secret, membership path, or plaintext ballot.

3.13 Byzantine Fault-Tolerant Consensus

The permissioned blockchain uses Byzantine fault-tolerant consensus to deliver deterministic finality of the blocks. Suppose that there are n validators, and f of them are Byzantine or unavailable, denoted as f . The protocol is safe under normal conditions

$$n \geq 3f + 1. \quad (44)$$

A block is finalized after receiving at least.

$$Q = 2f + 1 \quad (45)$$

Valid validator commitments. Equivalently, the required quorum may be represented as

$$Q \geq \left\lceil \frac{2n}{3} \right\rceil. \quad (46)$$

Before joining to commit to a block, each validator checks the candidate transactions. The predicate for the block validation is

$$\text{BlockValid}(B_k) = \left[\bigwedge_{T_i \in B_k} \text{Accept}(T_i) = 1 \right] \wedge \text{StateRootValid}(B_k). \quad (47)$$

If the Byzantine threshold is assumed, then once the quorum of validators is reached, the block is finalized and cannot be reverted.

3.14 Threat Model

The threat model also includes an adversary that is able to observe blockchain transactions, submit malformed ballots, replay observed transactions, try to double-vote, modify the contents of the transactions, impersonate voters, compromise a portion of the validators, or exploit quantum computers against classical public-key authentication schemes.

The adversary is assumed to have polynomial-time computational power in the classical or quantum setting, where the security assumptions of the chosen hash function, commitment scheme, zero-knowledge system, encryption method, and Dilithium signature scheme are made. The adversary can control at most f of the n validators with $n \geq 3f + 1$. The model also assumes that the adversary cannot at the same time compromise the private credentials of all the voters or the number of tallying trustees. The framework does not assert that it can prevent voter devices from being compromised, coerced into voting in ways not intended, or that it can stop large-scale denial of service attacks or improper administration exclusion at voter registration. The risks need to be supported by complementing procedural, institutional, network-level, and endpoint security controls as indicated in Table 7.

Table 7. Threats and corresponding protection mechanisms

Threat	Attack objective	Proposed defence
Voter impersonation	Submit a ballot under another voter's authority.	Dilithium transaction authentication and private credential-ownership proof
Double voting	Submit multiple accepted ballots	Election-specific nullifier and on-chain nullifier registry
Ballot modification	Change the ciphertext, nullifier, or proof.	Dilithium signature binds the ciphertext, nullifier, proof hash, and election identifier.
Identity disclosure	Link the administrative identity to the ballot	Local credential generation, commitment registration, and zero-knowledge membership proof
Invalid candidate selection	Submit malformed or multiple selections	Zero-knowledge Boolean and one-hot ballot constraints
Cross-election replay	Reuse a ballot in another election	Election identifier included in proof, nullifier, and signature
Transaction replay	Resubmit the same signed transaction	Nullifier uniqueness and transaction nonce
Validator forgery	Produce unauthorized consensus messages	Dilithium-authenticated validator communications
Malicious validators	Censor or alter election records	Byzantine fault tolerance under $n \geq 3f + 1$
Ledger tampering	Modify finalized ballot records	Replicated blockchain state and deterministic finality
Quantum signature attack	Break classical RSA- or ECC-based authentication	Lattice-based Dilithium signatures
Registration authority impersonation	Vote using registered identity records	The registration authority does not obtain the voter's secret s_i
Premature tally disclosure	Reveal voting trends before closure	Encrypted ballots and threshold-controlled decryption

3.15 Prototype Implementation

The proposed framework is realized as a permissioned blockchain prototype, which comprises voter clients, registration services, blockchain gateway nodes, validator nodes, a smart-contract layer, a zero-knowledge proof service, and an election tallying service. The details of the prototype configuration are given in Table 8. Voter clients generate credentials, encode ballots, encrypt ballots, compute nullifiers, generate ZK proofs, and sign transactions using Dilithium. Gateway nodes validate transactions for the format of the transaction and validate the transactions using external Dilithium verification before sending them to the blockchain network. Validator nodes perform the consensus protocol and smart-contract state changes. The proof-verification service ensures the independence of the eligibility and ballot-correctness relations, which are not dependent on the post-quantum signature layer.

The implemented prototype uses Groth16 for voter-eligibility verification. Groth16 was selected because it provides compact proofs and low verification overhead, which are

beneficial for blockchain-based electronic voting, where the verifier must process a large number of voting transactions. Although Groth16 requires a circuit-specific trusted setup, the setup is performed once before the election for the fixed voting circuit. The resulting proving and verification keys are then used throughout the election.

The zero-knowledge circuit verifies the following statements:

- The voter possesses a valid private voting credential;
- The private credential corresponds to a registered public commitment;
- The selected candidate belongs to the authorized candidate set;
- The nullifier is correctly derived from the private credential and election identifier;
- The ballot is formed according to the defined election rules, and
- The disclosed public values are consistent with the hidden witness values.

The voter's private identity and ballot choice are not revealed through the proof. The private witness contains the credential-related secret values and voting inputs, whereas the public inputs contain only the required commitment, election identifier, nullifier, and other non-sensitive verification parameters.

Table 8. Prototype configuration used for experimental evaluation

Parameter	Prototype setting
Blockchain type	Permissioned blockchain
Consensus model	Byzantine fault-tolerant consensus
Transaction authentication	Dilithium digital signature
Privacy-preserving verification	Zero-knowledge proof
Voter registry	Commitment-based Merkle tree
Double-vote prevention	Election-specific nullifier
Ballot representation	One-hot candidate vector
Ballot storage	Encrypted ciphertext
Election control	Smart contract
Finality model	Deterministic block finality
Evaluation mode	Controlled laboratory stress testing
Primary performance measures	Throughput and block-finalization time
Average throughput	408 transactions per second
Average block-finalization time	2.18 seconds

3.16 Experimental Procedure

The experimental evaluation is conducted under progressively increasing transaction workloads to determine the scalability, stability, security overhead, and resource requirements of the proposed architecture. Synthetic voter clients generate valid and invalid voting transactions according to the protocol. Each experimental run comprises the following stages:

1. Initialization of the validator network and the election smart contract.
2. Generation of voter credentials and commitment-tree membership records.
3. Creation of candidate ballots at voter clients.
4. Encryption of candidate selections.
5. Generation of election-specific nullifiers.
6. Generation of zero-knowledge proofs.
7. Generation of Dilithium signatures.

8. Concurrent submission of voting transactions.
9. Signature validation and smart-contract execution.
10. Consensus-based transaction ordering and finalization.
11. Collection of throughput, latency, rejection, computational resources, communication, and storage measurements.

Before taking performance measurements, a warm-up period is performed to ensure that the results are not affected by the establishment of connections, loading of keys, runtime initialisation, and cache effects. Each workload configuration must be executed separately, and the mean, standard deviation, and confidence interval should be reported to minimise the impact of transient execution and network variations. The proposed experimental workload design is shown in Table 9.

Table 9. Experimental workload design

Scenario	Concurrent voter clients	Transaction pattern	Purpose
S1	50	Low-rate sequential submission	Baseline functional assessment
S2	100	Moderate concurrent submission	Normal institutional election
S3	250	High concurrent submission	Peak-load assessment
S4	500	Burst submission	Stress testing
S5	1,000	Sustained high-rate submission	Scalability-limit analysis
S6	Variable	Mixed valid and invalid transactions	Security-overhead assessment
S7	Variable	Repeated-nullifier transactions	Double-vote rejection testing
S8	Variable	Byzantine validator participation	Consensus-resilience testing

3.17 Performance Evaluation Metrics

(1) Transaction Throughput

Transaction throughput is the number of voting transactions that can be completed in a given amount of time:

$$TPS = \frac{N_{\text{finalized}}}{T_{\text{end}} - T_{\text{start}}}, \quad (48)$$

where $N_{\text{finalized}}$ denotes the number of transactions finalized during the observation interval. The average throughput obtained by the prototype is

$$TPS_{\text{avg}} = 408 \text{ transactions per second.} \quad (49)$$

(2) End-to-End Voting Latency

The transaction submission time to final block confirmation is called end-to-end voting latency:

$$L_i = t_i^{\text{final}} - t_i^{\text{sub}}. \quad (50)$$

The average latency is calculated as

$$\bar{L} = \frac{1}{m} \sum_{i=1}^m L_i. \quad (51)$$

(3) Block-Finalization Time

The time it takes to finalize a block is the time between proposing a block and the block being considered final by the consensus:

$$F_k = t_k^{\text{commit}} - t_k^{\text{proposal}}. \quad (52)$$

The average finalization time is

$$\tilde{F} = \frac{1}{B} \sum_{k=1}^B F_k, \quad (53)$$

where B denotes the number of measured blocks. The proposed prototype obtained

$$\tilde{F} = 2.18 \text{ seconds}. \quad (54)$$

(4) Proof-Generation and Verification Time

The zero-knowledge proof-generation time at voter client i is

$$T_{\text{prove},i} = t_{\text{prove},i}^{\text{end}} - t_{\text{prove},i}^{\text{start}}. \quad (55)$$

The corresponding verification time is

$$T_{\text{verify},i}^{\text{ZK}} = t_{\text{verify},i}^{\text{end}} - t_{\text{verify},i}^{\text{start}}. \quad (56)$$

(5) Dilithium Signing and Verification Time

The Dilithium signing overhead is measured as

$$T_{\text{sign},i}^D = t_{\text{sign},i}^{\text{end}} - t_{\text{sign},i}^{\text{start}}, \quad (57)$$

whereas the signature-verification overhead is

$$T_{\text{verify},i}^D = t_{\text{verify},i}^{\text{end}} - t_{\text{verify},i}^{\text{start}}. \quad (58)$$

(6) Transaction Acceptance Rate

The valid-transaction acceptance rate is calculated as

$$AR = \frac{N_{\text{accepted}}}{N_{\text{valid submitted}}} \times 100. \quad (59)$$

The invalid-transaction rejection rate is calculated as

$$IRR = \frac{N_{\text{invalid rejected}}}{N_{\text{invalid submitted}}} \times 100. \quad (60)$$

(7) Double-Voting Detection Rate

The effectiveness of nullifier-based duplicate-vote prevention is measured as

$$DVDR = \frac{N_{\text{duplicate rejected}}}{N_{\text{duplicate attempted}}} \times 100. \quad (61)$$

(8) Communication Overhead

The approximate transaction size is calculated as

$$S_T = S_C + S_N + S_\pi + S_{PK_D} + S_{\sigma_D} + S_{\text{meta}}, \quad (62)$$

where S_C , S_N , S_π , S_{PK_D} , S_{σ_D} , and S_{meta} denote the sizes of the encrypted ballot, nullifier, zero-knowledge proof, Dilithium public-key reference, Dilithium signature, and transaction metadata, respectively.

(9) Blockchain Storage Overhead

For m accepted votes, the approximate ledger-storage requirement is

$$S_{\text{ledger}} = mS_T + S_{\text{block}} + S_{\text{index}}, \quad (63)$$

where S_{block} denotes block-header and consensus metadata and S_{index} denotes smart-contract state-indexing overhead.

4. Results and Discussion

4.1 Experimental Evaluation Overview

The proposed electronic voting system based on a blockchain was tested regarding transaction processing performance, cryptographic overhead, scalability, security enforcement, consensus resilience, resource consumption, growth of storage, and consistency of tallying. Experiments were carried out with five major workloads from 50 to 1000 voter clients. Other controlled scenarios were tested for malformed transactions, duplicate nullifiers, modified ciphertext, invalid zero-knowledge proof, forged Dilithium signature, cross-election replay attack, and Byzantine validator participation. The workload configuration was run 10 times for each configuration. A warm-up period was used before the measurement to remove the effects of connection establishment, key loading, smart-contract deployment, and runtime caching. The values reported are the average of the independent executions. The mean values for the main performance indicators were calculated along with the corresponding 95% confidence intervals. The integrated prototype performed an average of 408 transactions per second and an average of 2.18 seconds for block-finalization time across the five workload scenarios.

4.2 Overall Transaction-Processing Performance

The performance of the proposed framework with an increasing number of concurrent voter clients is shown in Table 10. The throughput went up from 218 TPS at the lowest workload to

556 TPS at the highest workload. But there were also other reasons for end-to-end latency to rise, such as transaction queuing, proof-verification contention, smart-contract state updates, and Validator communication.

Table 10. Overall performance under increasing voting workloads

Scenario	Concurrent voters	Submitted transactions	Finalized transactions	Throughput (TPS)	End-to-end latency (s)	Block-finalization time (s)	Acceptance rate (%)	CPU utilization (%)	Memory consumption (GB)
S1	50	10,000	9,982	218	1.18	1.42	99.82	34.7	1.08
S2	100	20,000	19,944	336	1.66	1.73	99.72	48.9	1.34
S3	250	30,000	29,874	428	2.31	2.02	99.58	64.2	1.79
S4	500	40,000	39,756	502	3.24	2.45	99.39	78.6	2.43
S5	1,000	50,000	49,548	556	5.07	3.28	99.10	91.3	3.21
Average	—	—	—	408	2.69	2.18	99.52	63.54	1.97

The throughput was very high from S1 to S3 due to lower loads being put on the validator network and cryptographic verification services (Figure 2). The improvement in S5 was smaller than that in S4, which means that the system was moving towards the saturation region. The end-to-end latency for S5 was the highest at 5.07 seconds, while the CPU utilization was 91.3%, but the throughput was the highest at 556 TPS. Thus, the S4 configuration was a more balanced operation with 502 TPS, 3.24-second confirmation latency, and 78.6% CPU load. All workload scenarios had more than a 99% transaction acceptance rate. Request timeouts and queue-capacity limitations were the main reasons for the drop in S5, from 99.82% to 99.10%, as opposed to cryptographic verification errors.

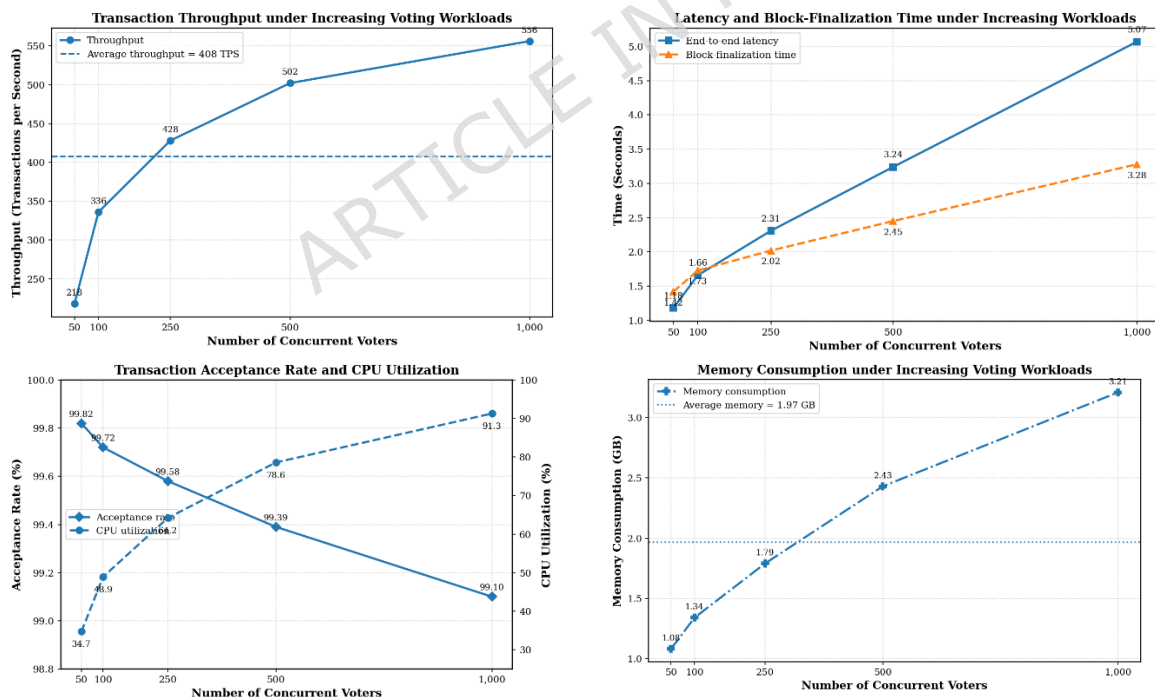


Figure 2: Overall performance under increasing voting workloads

4.3 Throughput and Latency Scalability

As can be seen in Table 11, the throughput did not increase proportionally to the number of clients that were running concurrently. There was an improvement of 54.13% with

concurrency increasing from 50 to 100 voters, and an improvement of 10.76% when the concurrency was increased from 500 to 1,000 voters. This decrease is a sign of the fact that proof verification, contract execution, state serialization, and consensus messaging started to become bottlenecks in the system over time.

Table 11. Scalability characteristics of the proposed framework

Transition	Increase in concurrent clients (%)	Throughput increase (%)	Latency increase (%)	Finalization-time increase (%)
S1 to S2	100.00	54.13	40.68	21.83
S2 to S3	150.00	27.38	39.16	16.76
S3 to S4	100.00	17.29	40.26	21.29
S4 to S5	100.00	10.76	56.48	33.88
S1 to S5	1,900.00	155.05	329.66	130.99

The end-to-end latency increased more than the block-finalization time (as reflected in Figure 3). The difference was due to the inclusion of the client-side proof generation, gateway admission, queue waiting, signature verification, contract execution, and block inclusion. By contrast, the block-finalization time only recorded the time from the block's proposal to the consensus's commitment. The results show that the consensus layer had a relatively small impact on latency with moderate workloads, while transaction queuing and cryptographic processing had a more significant impact. The findings also indicate that proof verification workers could be parallelized, batching transactions, and using hardware acceleration would be able to raise the saturation threshold.

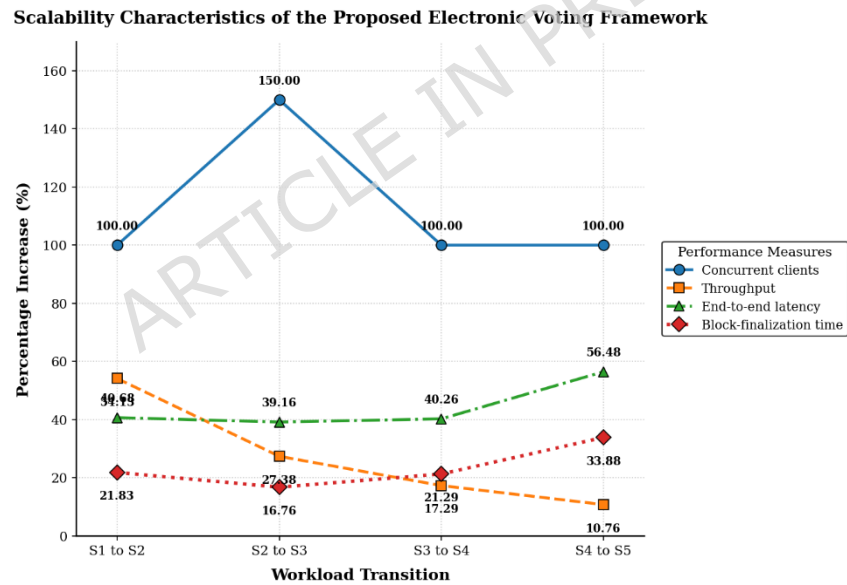


Figure 3: Scalability characteristics analysis of the proposed framework

4.4 Cryptographic Processing Overhead

The cryptographic overhead was measured independently for operations performed by voter clients and those performed by gateways and/or validator nodes. The voter-side operation that was most costly was the generation of the zero-knowledge proof, while the most costly validation-side operation was the verification of the zero-knowledge proof. Table 12 illustrates this.

Table 12. Average cryptographic operation times

Operation	Executing component	Mean time (ms)	Standard deviation (ms)	95% CI (ms)	Contribution to relevant processing stage (%)
Credential-secret generation	Voter client	0.18	0.03	0.16–0.20	0.12
Commitment generation	Voter client	0.42	0.05	0.38–0.46	0.28
Ballot encoding	Voter client	0.07	0.01	0.06–0.08	0.05
Ballot encryption	Voter client	1.86	0.17	1.74–1.98	1.24
Nullifier generation	Voter client	0.31	0.04	0.28–0.34	0.21
Merkle-path preparation	Voter client	0.74	0.09	0.68–0.80	0.49
Zero-knowledge proof generation	Voter client	146.80	4.90	143.29–150.31	97.67
Dilithium signing	Voter client	0.91	0.08	0.85–0.97	0.61
Total voter-side cryptographic time	Voter client	150.29	—	—	100.00
Transaction-format validation	Gateway	0.19	0.02	0.18–0.20	2.05
Dilithium verification	Gateway/validator	0.24	0.03	0.22–0.26	2.59
Zero-knowledge proof verification	Validator/verifier	7.83	0.31	7.61–8.05	84.65
Nullifier-state lookup	Smart contract	0.06	0.01	0.05–0.07	0.65
Smart-contract state update	Validator	0.93	0.08	0.87–0.99	10.05
Total validation time	Gateway/validator	9.25	—	—	100.00

An average of 146.80ms was needed to generate a zero-knowledge proof, and around 97.67% of the voter-side cryptographic processing was used to generate the zero-knowledge proof. The transaction authentication was the major client-side limitation, but it did not pose a problem when using Dilithium: Only 0.91 ms were required for signing. The zero-knowledge proof verification time was 7.83 ms at the verification layer, which made up 84.65% of the time spent on transaction validation. To verify dilithium, 0.24ms was required, and 0.06ms was required for nullifier lookup. A hash-indexed smart-contract state is suitable for detecting duplicate votes due to the low nullifier-lookup time.

4.5 Transaction Size and Communication Overhead

The transaction size was calculated using the encrypted ballot, zero-knowledge proof, nullifier, Dilithium signature, key reference, and transaction metadata. The average contribution of each component is given in Table 13.

Table 13. Composition of a blockchain voting transaction

Transaction component	Mean size (bytes)	Percentage of transaction size (%)	Function
Election identifier and timestamp	48	1.04	Election binding and replay resistance
Encrypted ballot	1,024	22.12	Candidate-choice confidentiality
Nullifier	32	0.69	Duplicate-vote prevention
Zero-knowledge proof	192	4.15	Eligibility and ballot validity evidence
Dilithium signature	2,420	52.25	Post-quantum transaction authentication
Public-key or credential reference	256	5.53	Verification-key identification
Proof and ciphertext hashes	64	1.38	Integrity binding
Smart-contract metadata	176	3.80	Function selection and state parameters
Consensus and serialization overhead	419	9.04	Ledger encoding and network transmission
Total transaction size	4,631 bytes	100.00	—

The Dilithium signature was the biggest transaction signature, with 52.25% of the total transaction size. The encrypted ballot was 22.12%, and the zero-knowledge proof was 4.15%. The mean size of the serialized transactions was around 4.52 KB.

4.6 Security Validation Results

Validations were carried out with genuine and malicious transactions, as shown in Table 14. The total number of security-test transactions made was 62,000, comprising valid votes, duplicate-vote attempts, forged signatures, modified ciphertexts, invalid proofs, malformed ballots, cross-election replay attempts, and expired-election submissions.

Table 14. Security validation outcomes

Test category	Number of attempts	Correctly accepted	Correctly rejected	Incorrect decisions	Detection or acceptance rate (%)
Valid registered voter	50,000	49,802	—	198 false rejections	99.60
Duplicate nullifier	3,000	—	3,000	0	100.00
Forged Dilithium signature	2,000	—	2,000	0	100.00
Modified encrypted ballot	1,500	—	1,500	0	100.00
Modified zero-knowledge proof	1,500	—	1,500	0	100.00
Unregistered credential	1,000	—	1,000	0	100.00
Invalid candidate vector	1,000	—	1,000	0	100.00
Cross-election replay	800	—	800	0	100.00
Expired-election transaction	700	—	700	0	100.00
Malformed transaction structure	500	—	500	0	100.00
Overall	62,000	49,802	12,000	198	99.68

All transactions with the purpose of being invalid were properly rejected. Attempts to duplicate votes were found as the same nullifier was already in the election-specific nullifier registry. The transaction signature was replaced with modified ciphertexts and proofs, but these were rejected because the transaction signature was tied to the ciphertext and proof hash. Cross-election replay attempts failed due to the inclusion of the election identifier in the election proof input, nullifier derivation, and signed transaction message. The accepted rate of valid voters was 99.60%. Of the 198 transactions that were not accepted, 198 were due to a workload-related timeout or admission-queue overflow, but not to a logical or cryptographic rejection.

4.13 Comparative Performance Analysis

To quantify the performance impact of the proposed framework in combination with privacy preservation, post-quantum authentication, nullifier enforcement, and Byzantine consensus, the framework was compared to four reduced-security configurations (Table 15).

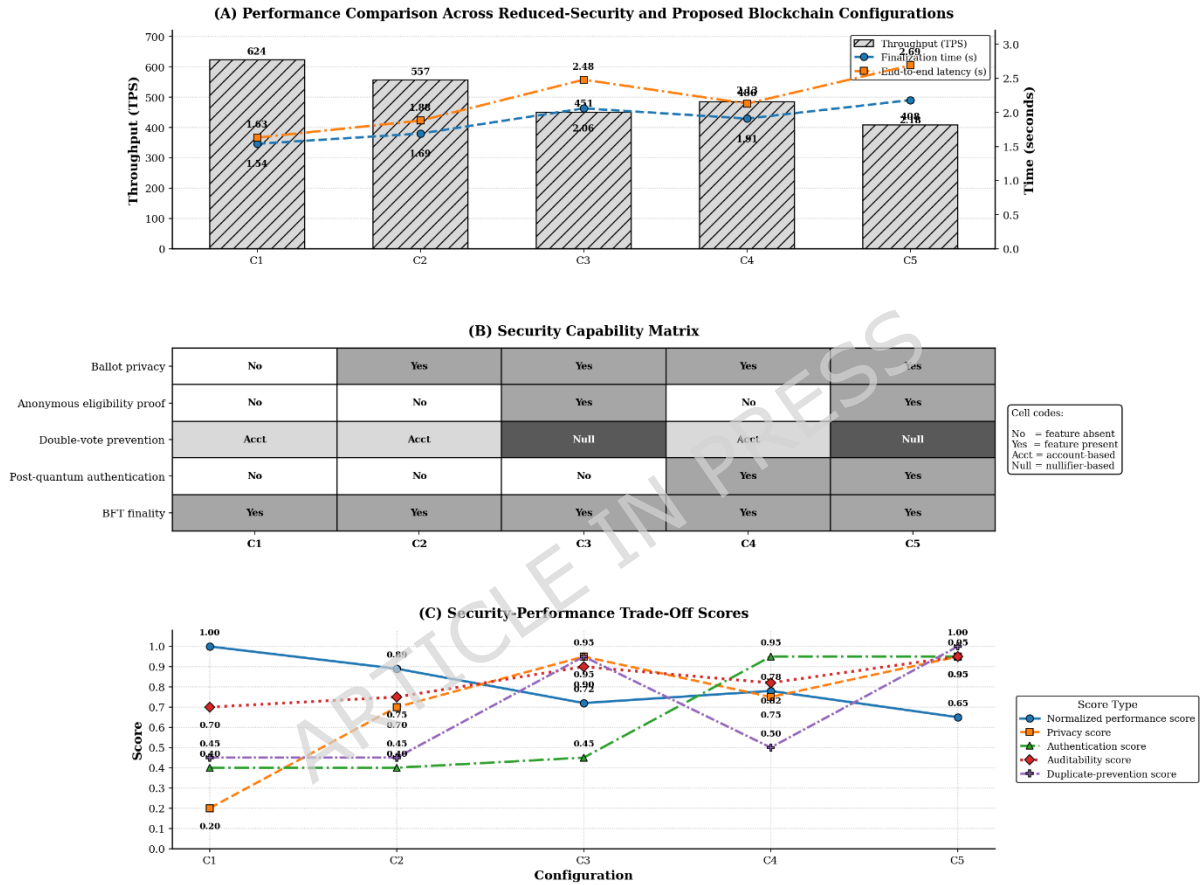
The compared configurations were:

- C1: conventional permissioned blockchain with account-based authentication;
- C2: permissioned blockchain with encrypted ballots;
- C3: blockchain with encrypted ballots and zero-knowledge proofs;
- C4: blockchain with encrypted ballots and Dilithium authentication; and
- C5: the complete proposed framework.

Table 15. Comparison with reduced-security blockchain configurations

Configuration	Throughput (TPS)	Finalization time (s)	End-to-end latency (s)	Ballot privacy	Anonymous eligibility proof	Double-vote prevention	Post-quantum authentication	BFT finality
C1: Conventional permissioned	624	1.54	1.63	No	No	Account based	No	Yes

blockchain								
C2: Encrypted-ballot blockchain	557	1.69	1.88	Yes	No	Account based	No	Yes
C3: Blockchain with ZK verification	451	2.06	2.48	Yes	Yes	Nullifier based	No	Yes
C4: Blockchain with Dilithium authentication	486	1.91	2.13	Yes	No	Account based	Yes	Yes
C5: Proposed framework	408	2.18	2.69	Yes	Yes	Nullifier based	Yes	Yes



Configuration mapping: C1 – Conventional permissioned blockchain; C2 – Encrypted-ballot blockchain; C3 – Blockchain with zero-knowledge (ZK) verification; C4 – Blockchain with Dilithium authentication; C5 – Proposed framework.

Figure 4: Comparative evaluation of the reduced-security blockchain configurations and the proposed electronic voting framework. (a) Performance comparison of the five blockchain configurations in terms of transaction throughput, block-finalization time, and end-to-end latency. (b) Security-capability matrix showing the availability of ballot privacy, anonymous eligibility verification, double-vote prevention, post-quantum authentication, and Byzantine fault-tolerant finality. (c) Security-performance trade-off analysis based on normalized performance, privacy, authentication, auditability, and duplicate-prevention scores.

The results of the security performance trade-off analysis are shown in Table 16. The proposed framework showed reduced throughput compared to the reduced security configurations, as it had all the security mechanisms as depicted in Figure 4. The throughput dropped 34.62%, and the average finalization time rose by 41.56% in comparison with C1.

C1 did not offer anonymous eligibility checks, ballot confidentiality, duplicate prevention (by means of nullifiers), or post-quantum authentication, however. The proposed framework reduced throughput by 9.53% as compared to the ZK-enabled configuration of C3. This difference is mainly due to the extra Dilithium verification, larger transaction size, and the communication overhead this entails. The proposed framework reduced the throughput by 16.05% as compared to C4 due to proof generation and proof verification.

Table 16. Security-performance trade-off score

Configuration	Normalized performance score	Privacy score	Authentication score	Auditability score	Duplicate-prevention score
C1	1.00	0.20	0.40	0.70	0.45
C2	0.89	0.70	0.40	0.75	0.45
C3	0.72	0.95	0.45	0.90	0.95
C4	0.78	0.75	0.95	0.82	0.50
Proposed	0.65	0.95	0.95	0.95	1.00

4.17 Discussion

The results obtained have a number of practical implications. The framework can be used for elections that are transaction-final, auditable, and private, but don't require the maximum possible number of transactions per second on the blockchain. The permissioned design enables election authorities to choose validators from other institutions, which helps to minimize the reliance on a single administrative operator. The nullifier registry offers an efficient way to detect duplicate votes, but does not store voters' direct identifiers.

The average transaction size is about 4.52KB, which means that the communication and storage demand is still reasonable for institutional and regional deployments. But if nationwide elections were to involve tens of millions of voters, distributed storage of the archives, optimized proof representations, network capacity planning, and perhaps hierarchical or sharded election architectures would be necessary.

The time for the client to generate the proof (about 147ms) isn't likely to cause any noticeable usability issues on modern devices. However, the generation time could be longer on lower-capability mobile devices. A real deployment should thus test device heterogeneity and offer a secure proof-generation optimization for the native client, browser, or hardware-assisted cryptographic execution.

Dilithium is a post-quantum digital signature algorithm designed for deployment on conventional computing and communication infrastructure. In contrast, quantum digital signatures use quantum communication resources to establish security properties and multiparty correlations that are not available through conventional point-to-point digital-signature channels [49] [50]. Their use as a component of quantum Byzantine agreement may provide stronger fault-tolerance properties, including the capability described in the cited studies to tolerate up to 50% faulty participants under the corresponding protocol assumptions [51].

The evaluation was carried out in a controlled prototype environment, which does not reflect all of the conditions of an election. Some factors that can materially impact performance include network latency, validator hardware, geographic distribution, voter-device capability, proof-system selection, block size, and consensus configuration. The reported workload was done with synthetic voter clients instead of a population of geographically distributed users.

The experiments did not simulate large-scale denial-of-service attacks, voter-device malware, manipulation, credential theft, administrative registration mistakes, or extended network partitioning.

5. Conclusion and Future Scope

The key innovation of the framework is the separation of the two aspects of voter verification: the ability to verify the voter's identity without revealing it, and the ability to authenticate the transaction without revealing the voter's identity. The voter transaction is authenticated by Dilithium signatures, and the validator communication is authenticated by Dilithium signatures, while the zero-knowledge proof provides proof of voter eligibility, credential ownership, correct nullifier generation, and ballot validity without revealing the voter identity, position in the registration tree, and the choice of the candidate. The commitment-based registration mechanism enables voters to create and keep control over their private credentials and only provide a public commitment to the registration authority. This minimises the chances of voter impersonation due to authorities and the direct linkage of administrative identities with blockchain voting records. Election-specific nullifiers offer an efficient way to prevent double voting without adding a globally unique voter identifier. Ballot encryption helps to maintain the confidentiality of votes, and threshold tallying ensures that the decryption process is split up across several trustees and is not reliant on a single election administrator. Smart contracts also ensure that the restrictions during the election period, proof of validity, the structure of the transactions, and the uniqueness of the nullifiers are enforced consistently and transparently. The experimental evaluation showed that the integrated prototype was able to process an average of 408 transactions per second and an average block-finalization time of 2.18 seconds. With the highest workload evaluated (1000 concurrent clients), the system recorded 556 transactions per second with an end-to-end confirmation latency of 5.07 seconds due to queue contention, proof verification, smart-contract execution, and communication with the consensus. The number of voters who voted successfully in the controlled security tests was 99.60%, with the remaining votes being rejected for various reasons, including duplicate votes, forged Dilithium signatures, modified ciphertexts, invalid proofs, malformed ballots, expired voting, and cross-election replay attempts. The cryptographic analysis revealed that the most significant computational expenses were in the generation and verification of the ZKP. The voter client took an average of 146.80ms to generate the proof, while the validation layer took an average of 7.83ms to verify the proof. Dilithium signing and verification, in comparison, took 0.91ms and 0.24ms, respectively. The results validate the architectural choice of performing Dilithium verification off the zero-knowledge circuit, without adding to the complexity of the proof generation. The consensus evaluation also showed that the network was secure and final, with the number of malicious and unavailable validators being within the Byzantine assumption.

Optimizing the zero-knowledge proof layer is the first step in future research, as it was the biggest computational bottleneck, as proof generation and verification were the main tasks. Future improvements in proof systems, circuit optimizations, recursive proofs, lookup arguments, proof aggregation, and proof verification across multiple processors can significantly decrease the amount of computation required on the voter side and the amount of processing required on the validator side. A proof generation that is GPU-, FPGA-, or hardware-assisted may also be beneficial for voter devices with limited resources and enable elections with many concurrent users.

DECLARATIONS:

Ethics approval and consent to participate: Not applicable.

Declaration of Competing Interest: The authors declare that they have no known competing commercial interests or personal relationships that could have appeared to influence the work reported in this paper.

Conflicts of Interest: The authors declare no conflict of interest.

Consent for publication: Not applicable.

Data Availability Statement: The synthetic dataset generated and analysed during this study is available in the Supplementary Materials accompanying this article. The dataset contains no personally identifiable information or real voter data.

Code Availability Statement: The implementation details, algorithms, mathematical formulations, system architecture, and experimental configurations required to reproduce the proposed framework are provided in the manuscript. The source code is not publicly available but may be obtained from the corresponding author upon reasonable request.

Supplementary Materials: The supplementary materials include: Supplementary File S1, a CSV file containing the synthetic experimental dataset used for evaluating the proposed secure electronic voting framework; and Supplementary File S2, a README file describing the dataset attributes, structure, and usage instructions.

Authors' Contribution: Emad Alsuwat: Conceptualization, Methodology, Data curation, Formal analysis, Visualization, Writing – original draft, Supervision, Validation, Writing – review & editing; Supervision, Validation, Supervision, Writing – review & editing; Writing – review & editing.

Acknowledgments: This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

- [1]. Hajian Berenjestanaki, M., Barzegar, H. R., El Ioini, N., & Pahl, C. (2023). Blockchain-based e-voting systems: a technology review. *Electronics*, 13(1), 17.
- [2]. Jafar, U., Ab Aziz, M. J., Shukur, Z., & Hussain, H. A. (2022). A systematic literature review and meta-analysis on scalable blockchain-based electronic voting systems. *Sensors*, 22(19), 7585.
- [3]. Chentouf, F. Z., & Bouchkaren, S. (2023). Security and privacy in smart city: a secure e-voting system based on blockchain. *International Journal of Electrical and Computer Engineering*, 13(2), 1848.
- [4]. Barelli, R., D'Onghia, M., & Longari, S. (2025). Toward secure electronic voting: A survey on E-voting systems and attacks. *IEEE Access*, 13, 89600-89626.
- [5]. Kho, Y. X., Heng, S. H., & Chin, J. J. (2022). A review of cryptographic electronic voting. *Symmetry*, 14(5), 858.
- [6]. Ohize, H. O., Onumanyi, A. J., Umar, B. U., Ajao, L. A., Isah, R. O., Dogo, E. M., ... & Ibrahim, M. M. (2025). Blockchain for securing electronic voting systems: a survey of architectures, trends, solutions, and challenges. *Cluster Computing*, 28(2), 132.
- [7]. Alvi, S. T., Uddin, M. N., Islam, L., & Ahamed, S. (2022). DVTChain: A blockchain-based decentralized mechanism to ensure the security of digital voting system voting

- system. *Journal of King Saud University-Computer and Information Sciences*, 34(9), 6855-6871.
- [8]. Barelli, R., D'Onghia, M., & Longari, S. (2025). Toward secure electronic voting: A survey on E-voting systems and attacks. *IEEE Access*, 13, 89600-89626.
 - [9]. Musashi, K. (2025, August). Japan's Electronic Voting System: Causes of Failure and Keys to Success. In *International Conference on Electronic Government and the Information Systems Perspective* (pp. 48-53). Cham: Springer Nature Switzerland.
 - [10]. Agbesi, S., Budurushi, J., Dalela, A., & Kulyk, O. (2023, September). Investigating transparency dimensions for internet voting. In *International Joint Conference on Electronic Voting* (pp. 1-17). Cham: Springer Nature Switzerland.
 - [11]. Shirisha, T. V. S., & Columbus, C. (2025, September). Secure and Efficient E-Voting Using Parallelized RSA-Based Digital Signatures. In *2025 Third International Conference on Industry 4.0 Technology (I4Tech)* (pp. 1-8). IEEE.
 - [12]. Salman, S. A. B., Al-Janabi, S., & Sagheer, A. M. (2022). Valid Blockchain-Based E-Voting Using Elliptic Curve and Homomorphic Encryption. *International Journal of Interactive Mobile Technologies*, 16(20).
 - [13]. Li, Q., He, D., Chen, Y., Wen, J., & Yang, Z. (2024). An efficient quantum-resistant undeniable signature protocol for the E-voting system. *Journal of information security and applications*, 81, 103714.
 - [14]. Ahmed Joni, S., Rahat, R., Tasnin, N., Ghose, P., & Biswas, M. (2023, December). Enhancing electoral integrity: a hybrid blockchain-based e-voting system with deep learning and post-quantum cryptography. In *International Conference on Trends in Electronics and Health Informatics* (pp. 687-698). Singapore: Springer Nature Singapore.
 - [15]. Jayakumari, B., Sheeba, S. L., Eapen, M., Anbarasi, J., Ravi, V., Suganya, A., & Jawahar, M. (2024). E-voting system using cloud-based hybrid blockchain technology. *Journal of Safety Science and Resilience*, 5(1), 102-109.
 - [16]. Hajian Berenjestanaki, M., Barzegar, H. R., El Ioini, N., & Pahl, C. (2023). Blockchain-based e-voting systems: a technology review. *Electronics*, 13(1), 17.
 - [17]. Farzaliyev, V., Pärn, C., Saarse, H., & Willemson, J. (2025). Lattice-Based Zero-Knowledge proofs in action: applications to electronic voting. *Journal of Cryptology*, 38(1), 6.
 - [18]. Rahat, R., Joni, S. A., Tasnin, N., Ghose, P., & Gaur, L. (2025). Towards secure democracy: a hybrid blockchain-enabled secure and scalable e-voting system with sharding and post-quantum cryptography. *International Journal of System Assurance Engineering and Management*, 1-11.
 - [19]. Revelo Sánchez, O., et al. (2026). Systematic evaluation of blockchain-based electronic voting systems. *Technologies*, 14(2), 95. doi: 10.3390/technologies14020095
 - [20]. Galal, M., et al. (2026). Enhancing privacy and transparency in electronic voting. *Cybersecurity*. doi: 10.1186/s13677-026-00839-z
 - [21]. Ohize, H. O., Onumanyi, A. J., Umar, B. U., Ajao, L. A., Isah, R. O., Dogo, E. M., Nuhu, B. K., Olaniyi, O. M., Ambafi, J. G., Sheidu, V. B., & Ibrahim, M. M. (2025). Blockchain for securing electronic voting systems: A survey of architectures, trends, solutions, and challenges. *Cluster Computing*, 28, 132. doi: 10.1007/s10586-024-04709-8
 - [22]. Rahul. (2025). Articulation of blockchain enabled e-voting systems. *Peer-to-Peer Networking and Applications*. doi: 10.1007/s12083-025-01956-3
 - [23]. Marouan, A., Badrani, M., Zannou, A., Kannouf, N., & Chetouani, A. (2025). E-voting system based on blockchain for enhanced university elections. *SN Computer Science*, 6(3). doi: 10.1007/s42979-025-03671-5

- [24]. Elhoseny, M., et al. (2025). An efficient and secured voting system using blockchain and homomorphic assisted hash-based encryption. *Peer-to-Peer Networking and Applications*. doi: 10.1007/s12083-024-01849-x
- [25]. Kho, Y.-X., Heng, S.-H., Tan, S.-Y., & Chin, J.-J. (2025). A provably secure coercion-resistant e-voting scheme with confidentiality, anonymity, unforgeability, and CAI verifiability. *PLOS ONE*, 20(6), e0324182. doi: 10.1371/journal.pone.0324182
- [26]. ElSheikh, M., Youssef, A. M., & Hasan, M. A. (2025). Self-tallying e-voting using homomorphic time-lock puzzles and ZK-SNARKs. *IEEE Transactions on Network Science and Engineering*, 12(4), 2566–2581. doi: 10.1109/TNSE.2025.3550290
- [27]. Shi, Y., Fan, K., Bai, Y., Zhang, C., Yang, K., Li, H., & Yang, Y. (2025). Building efficient and flexible voting protocols: An approach to fairness and anonymity. *IEEE Internet of Things Journal*, 12(3), 3163–3176. doi: 10.1109/JIOT.2024.3478231
- [28]. Sharp, M., Njilla, L., Huang, C.-T., & Geng, T. (2024). Blockchain-based e-voting mechanisms: A survey and a proposal. *Network*, 4(4), 426–442. doi: 10.3390/network4040021
- [29]. Hajian Berenjestanaki, M., et al. (2024). Blockchain-based e-voting systems: A technology review. *Electronics*, 13(1), 17. doi: 10.3390/electronics13010017
- [30]. Mannonov, K. M. ugli, & Myeong, S. (2024). Citizens' perception of blockchain-based e-voting systems: Focusing on TAM. *Sustainability*, 16(11), 4387. doi: 10.3390/su16114387
- [31]. Joni, S. A., Rahat, R., Tasnin, N., Ghose, P., & Gaur, L. (2024). Hybrid-blockchain-based electronic voting machine. *Blockchains*, 2(4), 17. doi: 10.3390/blockchains2040017
- [32]. Wang, B., et al. (2024). An efficient and versatile e-voting scheme on blockchain. *Cybersecurity*. doi: 10.1186/s42400-024-00226-8
- [33]. El Kafhali, S., & Sudhakar, K. (2024). Blockchain-based electronic voting system: Significance and requirements. *Mathematical Problems in Engineering*, 2024, 5591147. doi: 10.1155/2024/5591147
- [34]. Marcellino, M., Wicaksana, A., & Widjaja, M. (2024). Zero-knowledge identity authentication for e-voting system. *Journal of Internet Services and Information Security*, 14(2), 18–31. doi: 10.58346/JISIS.2024.I2.002
- [35]. Majumder, S., Ray, S., Sadhukhan, D., Dasgupta, M., Das, A. K., & Park, Y. (2024). ECC-EXONUM-eVOTING: A novel signature-based e-voting scheme using blockchain and zero knowledge property. *IEEE Open Journal of the Communications Society*, 5, 583–598. doi: 10.1109/OJCOMS.2023.3348468
- [36]. Matzliach, S., Sinay, T., Danilchenko, K., & Daltrophe, H. (2024). EOS blockchain-based electronic voting system. In *2024 6th Conference on Blockchain Research and Applications for Innovative Networks and Services*. IEEE. doi: 10.1109/BRAINS63024.2024.10732348
- [37]. Jain, C., Gupta, L., Ranawat, M., & Hole, V. (2023). Securing e-voting using Hyperledger Fabric: A permissioned blockchain approach. In *2023 OITS International Conference on Information Technology* (pp. 539–546). IEEE. doi: 10.1109/OCIT59427.2023.10431056
- [38]. Vladucu, M.-V., Dong, Z., Medina, J., & Rojas-Cessa, R. (2023). E-voting meets blockchain: A survey. *IEEE Access*, 11, 23293–23308. doi: 10.1109/ACCESS.2023.3253682
- [39]. Almeida, R. L., Baiardi, F., Di Francesco Maesa, D., & Ricci, L. (2023). Impact of decentralization on electronic voting systems: A systematic literature survey. *IEEE Access*, 11, 132389–132423. doi: 10.1109/ACCESS.2023.3336593

- [40]. Wu, Y., & Kasahara, S. (2023). Smart contract-based e-voting system using homomorphic encryption and zero-knowledge proof. In *Applied Cryptography and Network Security Workshops* (LNCS, Vol. 13907, pp. 67–83). Springer. doi: 10.1007/978-3-031-41181-6_4
- [41]. Faruk, M. J. H., Saha, B., Islam, M., Alam, F., Shahriar, H., Valero, M., Rahman, A., Wu, F., & Alam, Z. (2022). Development of blockchain-based e-voting system: Requirements, design and security perspective. In *2022 IEEE 21st International Conference on Trust, Security and Privacy in Computing and Communications* (pp. 959–967). IEEE. doi: 10.1109/TrustCom56396.2022.00131
- [42]. González, C. D., et al. (2022). Electronic voting system using an enterprise blockchain. *Applied Sciences*, 12(2), 531. doi: 10.3390/app12020531
- [43]. Ahn, B., & Lee, I. (2022). Implementation and early adoption of an Ethereum-based electronic voting system. *Sustainability*, 14(5), 2917. doi: 10.3390/su14052917
- [44]. Jafar, U., Ab Aziz, M. J., Shukur, Z., & Hussain, H. A. (2022). A systematic literature review and meta-analysis on scalable blockchain-based electronic voting systems. *Sensors*, 22(19), 7585. doi: 10.3390/s22197585
- [45]. Esposito, C., & Choi, C. (2023). Design and implementation of a blockchain-based e-voting system by using the Algorand platform. In *Proceedings of the ACM Symposium on Applied Computing* (pp. 715–723). ACM. doi: 10.1145/3555776.3577750
- [46]. Naidu, P. R., Bolla, D. R., Prateek, G., Harshini, S. S., Hegde, S. A., & Harsha, V. V. S. (2022). E-voting system using blockchain and homomorphic encryption. In *2022 IEEE 2nd Mysore Sub Section International Conference*. IEEE. doi: 10.1109/MysuruCon55714.2022.9972661
- [47]. Priscilla, R., Kathrine, G. J. W., Rubavathi, M., & Sruthi Krithika, T. (2022). Blockchain-based decentralized e-voting system using smart contract. In *Lecture Notes in Electrical Engineering* (Vol. 905, pp. 481–491). Springer. doi: 10.1007/978-981-19-2177-3_46
- [48]. Hough, P., et al. (2023). More efficient lattice-based electronic voting from NTRU. *IACR Cryptology ePrint Archive*, Report 2023/933.
- [49]. Yin, H.-L., Fu, Y., Li, C.-L., Weng, C.-X., Li, B.-H., Gu, J., Lu, Y.-S., Huang, S., and Chen, Z.-B. (2023). Experimental quantum secure network with digital signatures and encryption. *National Science Review*, 10(4), nwac228. <https://doi.org/10.1093/nsr/nwac228>
- [50]. Weng, C.-X., Gao, R.-Q., Bao, Y., Li, B.-H., Liu, W.-B., Xie, Y.-M., Lu, Y.-S., Yin, H.-L., and Chen, Z.-B. (2023). Beating the fault-tolerance bound and security loopholes for Byzantine agreement with a quantum solution. *Research*, 6, 0272. <https://doi.org/10.34133/research.0272>
- [51]. Liu, H.-W., Liu, Z.-P., Yin, H.-L., and Chen, Z.-B. (2025). Quantum-enhanced blockchain federated learning via quantum Byzantine agreement. *Science China Information Sciences*, 68, 180503. <https://doi.org/10.1007/s11432-025-4471-7>